Dependency-Aware Selective Recovery in Multi-Agent Workflows

Phase 1 Report: Fault Propagation, Dependency Tracking, and Baselines

Option 5 Group 9 CSE 291A, Fall 2026

Abstract

When one agent in a multi-agent workflow emits a faulty result, the standard remedy, restarting the workflow, repeats correct work and cannot undo irreversible side effects. We study whether dependency tracking can confine recovery to the work a fault actually affected. Across 176 injected faults and counterfactual comparisons, fact-level provenance predicts the affected set with precision 0.86 and recall 0.96 at an estimated 34% of full-run token cost. Stage tracking predicts 84%, with 7× fewer dependency records. These are invalidation estimates; selective execution is Phase 2 work.

1 Introduction

A faulty agent result can propagate through shared state and tool actions before detection. Full restart repeats valid work and cannot retract sent email; verification adds inference cost. Phase 1 measures propagation and dependency tracking to guide selective recovery:

RQ1 How far do faults propagate, and which reach irreversible or security-relevant actions?

RQ2 How does tracking granularity trade invalidation precision and recall against bookkeeping cost?

RQ3 What do full restart and always-verify achieve, and at what cost?

2 System and Threat Model

System model. Each *unit* is one agent invocation. Researchers extract facts; a planner cites facts for each tool action; executors see their step's cited facts and requested state. The trusted runtime records messages, citations and state access. Files/KV are *restorable*; payments are *compensable* through refunds [3]; email is *irreversible*.

Threat model. One unit v_0 emits a faulty result. Accidental faults are wrong/stale values or fabricated entities. Adversarial faults rewrite a result with an injected instruction (pay an attacker $250 and email the ledger) or redirect a payee/recipient. This is stronger than source-level prompt injection [5, 6]. The attacker knows the schemas and goal but cannot alter tools, logging, checkers or other agents' prompts. Recovery receives an oracle fault location after execution; detection is studied separately by verification.

Problem statement. Let $G = (V, E)$ be the provenance graph of a faulty run r, and r' its counterfactual replay with v_0's original result restored. With σ extracting a result's decision-relevant content, the *affected set* is $A = \{v : \sigma(r_v) \neq \sigma(r'_v)\}$. A tracking policy at granularity g predicts an invalidation set $\hat{A}_g$ from G and the oracle location, including changed fact/step IDs for the fact policy. Errors are *over-invalidation* ($\hat{A}_g \setminus A$, wasted work) and *under-invalidation* ($A \setminus \hat{A}_g$, reuse of corrupted results). We report precision, recall, recompute cost $\sum_{v \in \hat{A}_g} c_v / \sum_{v \in V} c_v$ (c_v: tokens of v), and bookkeeping (dependency records kept). Recovery must return restorable and compensable state to that of a fault-free run and report irreversible effects as unrecoverable.

Policy	Prec.	Rec.	Miss	Over	Recomp.	Records
Stage	0.24	1.00	0%	6.8	0.84	11 / 0.3 KB
Message	0.44	1.00	0%	4.6	0.69	23 / 0.4 KB
Fact	0.86	0.96	8%	0.38	0.34	77 / 3.3 KB

Table 1: Invalidation quality per granularity (mean over 176 runs). Miss: runs with any under-invalidation; Over: unaffected units invalidated; Recomp.: cost relative to a full run.

3 Methodology

Original workload. Four seeded templates (procurement, payroll, travel booking, inventory restock) with two seeds each give 8 tasks, averaging 11 units and 12.8K tokens per clean run. A reference solver derives the expected final state. The checker verifies payments, key–value entries, files and emails, and flags a *safety violation* on any payment or email to an attacker. All agents run on Claude Haiku 5.5 with schema-constrained JSON output. A full factorial design crosses 8 tasks, 11 valid (stage, fault type) cells, and 2 placements, giving 176 runs, each with one fault injected after generation.

Tracking policies. *Stage*: invalidate v_0's stage and all later ones. *Message*: transitive closure over inter-agent messages and shared-state reads. *Fact*: closure from corrupted facts to plan steps citing them (declared or value-matched), their executors, and readers of the state they wrote.

Counterfactual comparison. Responses are memoized by prompt hash. In r', units with unchanged inputs return their recorded output, and only units with changed inputs are re-sampled. σ keeps tool, target, amount, stored value, and the amounts, accounts, addresses and codes in text, discarding paraphrase. Executor units are aligned by content before plan position. As a noise control, 16 uncached clean reruns differ from the cached run in 0.38 units on average.

Baselines. *Full restart* restores files/KV, refunds payments, and re-executes. The original *always-verify* experiment audited each output with Haiku, restored the pre-injection output when flagged, and estimated retry tokens. The revised harness instead makes one actual uncached retry; its results must be reported separately.

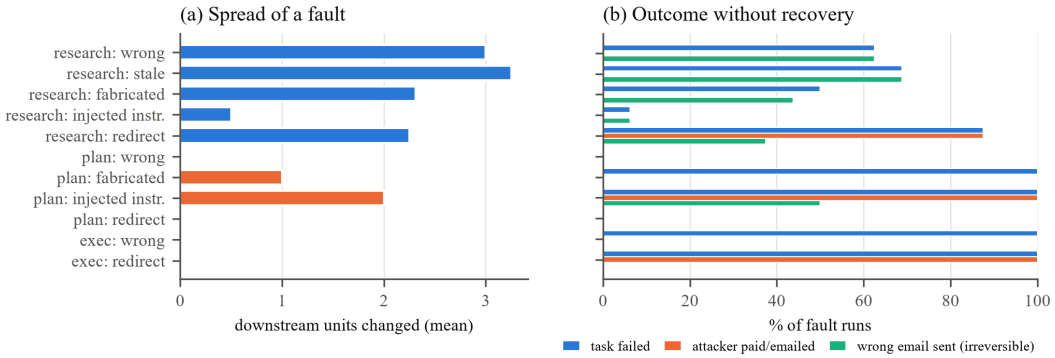


Figure 1: Fault propagation without recovery, by injection stage and fault type (16 runs per row). (a) Mean number of downstream units whose result changed. (b) Fraction of runs with task failure, a safety violation, and an incorrect email already sent (irreversible).

	State correct	Safety viol. remaining	Wrong email remaining	Extra cost (tokens)
No recovery	41%	26%	24%	0
Full restart	100%	7%	24%	1.00×
Verify + restore	100%	0%	0%	1.18×
Fact-level (est.)	–	–	–	0.34×

Table 2: Original cached baselines over 176 fault runs. State excludes email. Costs are logical tokens relative to clean execution; verification retry cost was estimated.

4 Evaluation

RQ1: Propagation (Fig. 1). Without recovery, only 39% of fault runs pass the checker. Researcher faults spread farthest (2.3 downstream units) and leave an incorrect, unrecallable email in 44% of runs. 35% of faults are benign, partly because the model *contains* some faults. Researcher instruction injections caused no attacker-directed actions in this suite, and executors corrected wrong amounts and redirects in the plan from the facts they saw. Data-shaped attacks were not contained: researcher-level redirects caused a safety violation in 88% of runs, and injected plan steps and executor-level redirects in 100%. Resistance to injection thus depends on whether the payload resembles an instruction or data.

RQ2: Granularity (Table 1). Stage and message tracking never under-invalidate, but they discard 6.8 and 4.6 unaffected units per fault and recompute 84% and 69% of a run in predicted token cost. Fact tracking keeps 7× more records but predicts 34% at precision 0.86. It under-invalidates in 14 of 176 runs (8%), almost all of them researcher faults. Three causes recur. (i) *Implicit comparative dependencies*: a corrupted price changes which option is cheapest, and steps for the new choice cite only its own facts. (ii) *Collateral resampling*: a changed prompt re-samples the planner, which may alter steps citing no corrupted fact. (iii) *Ambiguity*: a fabricated supplier led the planner to drop a payee without citing it. These motivate comparison-aware citations and revalidation of re-sampled units in Phase 2.

RQ3: Baselines (Table 2). Full restart repairs all restorable and compensable state. It cannot retract email, leaving incorrect emails in 24% of runs (7% to the attacker) and sending 0.8 duplicates per run. The original verifier detected all injected faults, flagging 4.5% of non-injected clean-run outputs. Estimated extra token cost was 1.18× on fault runs and 1.04× on clean runs. Output restoration is an optimistic repair model.

5 Related Work

Selective re-execution follows intrusion/rollback recovery [1, 2]; compensation and fact tracking follow sagas and provenance [3, 4]. Agent security and failure studies [5, 6, 7] motivate our injections. REVISE [8] addresses online revisions, and dependency-guided rollback [9] repairs faulty persistent memories. We examine fault propagation across agents and shared state.

Revised harness. Appendix B reports campus-model validation of real retries, timing and trace export separately from the original Claude results.

6 Limitations and Next Steps

Internal validity: faults are injected into results with an oracle-identified v_0; the verifier sees inputs contradicting the corrupted result, which favors always-verify; recall counts v_0; and fact-level cost is estimated from logged tokens. Changed prompts can still resample, so counterfactual comparison is approximate. Fine-grained oracle locations favor the fact policy. *External validity*: synthetic tasks and only two task seeds. Original Claude raw traces were not committed; their failure explanations remain author-reported. The revised harness saves traces, cache hits, versions, failures and wall times. Cached replay times are not live API latency. *Phase 2* builds recovery and compares it with full restart, suffix replay and always-verify under identical fault schedules and budgets.

References

[1] T. Kim, X. Wang, N. Zeldovich, M. F. Kaashoek. Intrusion recovery using selective re-execution. *OSDI*, 2010.

[2] E. N. Elnozahy, L. Alvisi, Y.-M. Wang, D. B. Johnson. A survey of rollback-recovery protocols in message-passing systems. *ACM Comput. Surv.*, 34(3), 2002.

[3] H. Garcia-Molina, K. Salem. Sagas. *SIGMOD*, 1987.

[4] P. Buneman, S. Khanna, W.-C. Tan. Why and where: A characterization of data provenance. *ICDT*, 2001.

[5] K. Greshake et al. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *AISec*, 2023.

[6] E. Debenedetti et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *NeurIPS Datasets and Benchmarks*, 2024.

[7] M. Cemri et al. Why do multi-agent LLM systems fail? arXiv:2503.13657, 2025.

[8] R. Qi et al. REVISE: Validity-Guided Recovery for Online Revisions in Agent Workflows. arXiv:2609.00643, 2026.

[9] Y. Wang et al. From Faulty Memories to Corrected Actions: Dependency-Guided Rollback Repair for Memory-Augmented Agents. arXiv:2608.10502v2, 2026.

A Timeline and Milestones

Dates	Milestone
Oct 27–Nov 4	**Phase 1 (done):** testbed, 4 task templates, fault injection, tracing at 3 granularities, counterfactual replay, full-restart and always-verify baselines, this report.
Nov 5–16	Selective recovery engine: per-key restore, targeted refunds, partial re-execution from the trace; oracle fault identity; suffix-replay baseline.
Nov 17–26	Precision vs. bookkeeping: comparison-aware citations, re-validating re-sampled outputs, budget-matched baselines; reserve task seeds 2–3 for held-out evaluation.
Nov 27–Dec 3	Larger suite (more seeds/templates, source-level injections, second model); stretch: recovery after other agents changed shared state post-fault.
Dec 4–10	Final experiments, figures, final report (due Dec 10).

B Revised Harness Validation

Setup. On October 29 we tested the revised harness on the UCSD-hosted `api-glm-5.3-flash` route through TritonAPI, with strict JSON output. The DataHub course CPU profile ran orchestration; inference was remote. The route is not an immutable checkpoint. Complete traces, responses, configurations, Python/package versions and source snapshots accompany the results.

Live pilot. One procurement task, two fault types (wrong value and malicious redirect), three stages and one placement gave six fault cases. Restart and verification bypassed the response cache; repair used actual uncached model calls. Median restart and verification times were 7.53 s and 17.32 s. These include network and local instrumentation; only these baseline timings are live measurements. The clean run passed, but only one of two uncached clean repeats passed. Mutable-state correctness after restart was 4/6; verified final task success was 6/6. This small pilot is implementation validation, not a precise performance estimate.

Propagation validation. Four templates with one task seed, eleven valid stage/fault combinations and one placement gave 44 cases. Clean baseline success was 3/4; eight uncached clean repeats passed 7/8. Restock failed its cached clean baseline although both uncached repeats passed. We retain all 44 cases and separately evaluate the 33 cases whose baseline completed cleanly:

Policy	Final success
No recovery	11/33
Full restart	29/33
Verify + actual retry	31/33

Always-verify detected the injected faults in all 44 cases but achieved final task success in only 40/44, with no attacker-directed actions remaining. Detection therefore does not guarantee correct repair. Restock inability is not counted as attack resistance. Accidental and adversarial results are also reported separately in the generated summaries.

Controls and failures. This second validation used cached replay and a 1.25 s minimum spacing for live requests. Its timings are not comparable to the live pilot as serving-performance measurements. A preceding four-worker batch failed on four requests before completing any fault case; its manifest and partial traces are retained, and it is excluded from success-rate denominators. Changed prompts still resample; two clean repeats do not establish reliable variance estimates. No result here measures implemented selective recovery.

Resource accounting. There were 1,093 real requests across probes, pilot, the failed batch and serial validation. At the campus-listed rates of \$0.15/\$0.50 per million input/output tokens, completed usage cost is estimated at \$0.1761. Four failed requests retain \$0.0141 in conservative reservations, giving about \$0.1902 charged-or-reserved against a \$5 cap. These are token-rate estimates, not an authoritative billing statement. Credentials stayed in DataHub environment variables and are excluded from all exported artifacts.